

Scalability & Future Plan

Date	15 Julyv2026
Team ID	
Project Name	Rising Waters: Smart AI Flood Prediction System
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	Static Dataset Training Bounds: Predictions are reliant on bounded historical parameters and may drop in accuracy during black swan weather events.	Reduces flexibility when encountering historical climatological anomalies.	Medium
2	Manual Data Ingestion: Form fields require users to manually type in rainfall and atmospheric features.	Slows down prompt advisory issuance during critical flash-flood events.	High
3	Single Server Runtime Container: The default local Flask web development runtime is single-threaded and lacks multi-instance load balancing.	High risk of system lag or downtime if targeted by a sudden spike in concurrent regional users.	High

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Handles ~50 simultaneous connections smoothly before resource starvation triggers latency blocks	Migrate runtime orchestration onto IBM Cloud Code Engine with autoscale triggers set to replicate instances when CPU loads pass 70%.
2	Data Storage	Local folder serialization tracking via joblib files (floods.save) with no historical database logging.	Integrate an active cloud relational data instance (PostgreSQL/IBM Cloudant) to record live telemetry requests permanently.
3	Performance	Basic synchronous Flask worker routing blocks processing threads when analyzing massive matrix arrays.	Deploy a WSGI-compliant application gateway stack (Gunicorn) alongside multi-core worker nodes to ensure non-blocking task cycles.
4	Security	Basic client-side form validations without secure API rate limits or input sanitization firewalls.	Implement robust schema validation layers (Marshmallow) alongside web access token protection middleware to block brute injection attacks.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Asynchronous Model Caching Integration	1 Month	Eliminates redundant disk reads by loading the model directly into persistent application memory states.
Phase 3	Automated Radar Telemetry API Webhooks	3 Months	Drastically cuts response times by pulling automated live climate matrices directly into the inference engine.
Phase 4	Geographic Spatial Hotspot Mapping Layer	6 Months	Empowers field rescue teams by visually rendering active high-risk sectors directly onto an interactive frontend map.